

SonicBolt 输入双帧 Ping-Pong 缓存改造分析

1. 结论先行

如果目标是：

- 在 **不大改现有流协议** 的前提下加入输入双帧缓存
- 让 **下一帧输入装载** 与 **当前帧卷积计算** 并行
- 尽量少改模块、少加端口、少动代码

那么我建议本轮 **只改 3 个 RTL 模块**：

1. `cnn.v`
2. `conv/conv_subsystem.v`
3. `conv/conv_shared_input_buffer.v`

其中真正的主要改动集中在：

- `conv_shared_input_buffer.v`：把当前单帧输入缓存改成双 bank
- `conv_subsystem.v`：增加一个很小的“下一帧发车控制器”

而下面这些模块，本轮 **不建议修改**：

- `conv/conv_core.v`
- `conv/conv_param_store.v`
- `dwconv/*`
- `pwconv/*`
- `post_process/*`

核心原因很简单：**当前系统真正的单帧瓶颈只在 Conv 输入侧的整帧存储**，不是在层间 token 流协议本身。

2. 整体架构设计思路

2.1 推荐思路

推荐把“Ping-Pong”只放在 **Conv 输入整帧缓存** 这一层，不把 `frame_id`、`bank_id` 一路传到 DWConv / PWConv / Post-Process。

也就是说：

- `valid / last / pos / group / fire` 这套层间流协议 **保持不变**
- 只在 Conv 输入前端增加 **双 bank 输入图缓存**
- 当前帧从 bank A 读
- 下一帧同时写入 bank B
- 当前帧 Conv 结束后，立即切换到 bank B

这样做的优点是：

1. 改动集中在最前端，不会把新元数据扩散到整条流水线。
2. 现有 DWConv / PWConv / MaxPool / FC 的接口宽度都不用改。
3. 现有验证通过的主数据通路基本不动，风险最低。

2.2 双帧缓存到底要双什么

本轮不需要把 `conv_shared_input_buffer` 里的所有状态都双份化。

建议双份化的只有：

- 整帧输入 SRAM bank
- `pos=0` 启动时用到的 `shadow_cache`

建议继续保留单份的：

- 当前 14 行工作集 `pos_window_data`
- 两条预取行 `prefetched_rows`
- `current_pos / prefetch_step / workset_loaded` 这些“正在消费当前帧”的状态

原因是：

- Conv 同一时刻仍然只会消费一帧
- 真正需要并行的是“**一边算当前帧，一边写下一帧**”
- 所以只要把“整帧存储体”做成双 bank 即可
- 工作集和预取寄存器仍然只服务当前 active bank，没必要复制两套

这能明显减少新增代码量和面积增量。

2.3 推荐的帧切换方式

既然采用方案 B，那么推荐把 bank 管理完全收敛到内部：

- 外部只负责逐行写入一张图
- 外部在整帧写完后拉高一个 `img_wr_commit`
- `conv_shared_input_buffer` 内部自己决定“这张图被写进哪个 bank”
- 并把该 bank 标成 ready
- 当 `conv_core` 空闲，且存在 ready bank 时，`conv_subsystem` 自动产生一拍 `launch_pulse`
- 同时告诉 `conv_shared_input_buffer`：“下一帧从哪个 ready bank 开始消费”

这样一来：

- 外部不需要知道 `conv_core` 什么时候空闲
- 顶层也不需要额外暴露 `conv_busy` 或 `start_ready`
- 能避免因为 `cnn.v` 当前 `busy = conv || dwconv || pwconv || post` 而错过最早启动下一帧的时机

这是本方案里一个很关键的点。

3. 为什么只改 Conv 输入侧

3.1 当前单帧瓶颈确实在 `conv_shared_input_buffer`

从代码看，当前 `conv_shared_input_buffer.v` 明确是：

- 一套单口输入 SRAM
- 一套 `shadow_cache`
- 一套工作集

并且注释里已经写明“当前版本不再保留双 bank ping-pong”。

因此当前结构下：

- 计算当前帧时，输入图读和下一帧写无法真正并行
- 下一帧必须等当前帧输入缓存释放后再装
- 帧与帧之间必然有输入装载黑洞

3.2 后续层并没有这个“整帧输入缓存冲突”

DWConv / PWConv / Post-Process 当前都是围绕 token stream 工作：

- DWConv：直接吃 Conv 流输出
- PWConv：只做 **pos 级** odd/even 双缓冲
- Post-Process：也是流式处理，没有整帧输入图 SRAM

所以本轮要解决的“输入双帧缓存”问题，本质上不在这些模块里。

3.3 不建议本轮给整条流水加 frame_id

如果给 out_stream_* 全链路增加 frame_id，代价会很高：

- cnn.v
- conv_core.v
- dwconv_core.v
- pwconv_core.v
- maxpool.v
- fc.v
- meta_pipe.v
- rescale_relu.v

这些模块的接口和对齐逻辑都要改。

这和“尽量不要增加过多代码”的目标相反，所以本轮不推荐。

4. 建议修改的模块与原因

4.1 conv/conv_shared_input_buffer.v

这是 **必须重构** 的模块，也是本轮改造的核心。

需要增加的功能

1. 把单帧 frame_store 改成双 bank。
2. 把 shadow_cache[0:13] 改成双 bank 版本。
3. 为每个 bank 增加 ready 状态位。
4. 增加内部 write_bank 分配逻辑。

- 5. 增加“当前消费 bank”选择逻辑。
- 6. 对外提供“当前还能不能继续写”的状态反馈。
- 7. 保持现有 pos_window_data + prefetched_rows 为单份 active 工作集。

建议新增端口

端口名	方向	位宽	作用
img_wr_commit	input	1 bit	表示当前正在写入的这一帧 30 行已经写完，可以被消费
consume_bank_sel	input	1 bit	本次 start_consume 要切到哪个 ready bank
ready_bank_mask	output	2 bit	输出给 conv_subsystem，指示哪几个 bank 已 ready
img_wr_ready	output	1 bit	输出给外部，表示当前存在可写 bank，可以开始/继续写一张新图

为什么要这样改

因为这个模块现在既承担：

- 整帧输入图存储
- pos=0 的快速建窗
- 后续 pos 的滑窗预取

所以双帧能力必须从这里长出来。

但又因为一次只消费一帧，所以工作集和预取状态不用双份化，只要：

- 双 bank 存整帧
- 单 active workset 做当前帧滑窗

就够了。

具体实现建议

建议内部状态至少包含：

- active_bank
- write_bank
- write_inflight
- ready_bank[1:0]

- shadow_cache_bank0[0:13]
- shadow_cache_bank1[0:13]

并把 SRAM 读写路径改成：

- 写路径：按内部 write_bank 写对应 bank
- 读路径：永远只从 active_bank 读

这样就能天然避免“当前帧读”和“下一帧写”打到同一单口 SRAM。

更具体地说，建议这样处理写入会话：

- 当检测到一张新图开始写入时，内部锁定一个 write_bank
- 后续 addr=0..29 的 30 行都写到这个 write_bank
- 收到 img_wr_commit 后，把这个 write_bank 标成 ready
- 然后再为下一张图重新选择可写 bank

如果你们当前外部写入流程能保证每张图都按 addr=0..29 顺序写入，那么第一版可以直接用：

- img_wr_en && (img_wr_addr == 0) 作为“新一帧写入开始”的判据

如果后续希望接口更健壮，再额外补一个 img_wr_frame_start 也可以，但本轮不是必需。

4.2 conv/conv_subsystem.v

这个模块建议做 **小改**，但它是整个方案成立的关键胶水层。

需要增加的功能

1. 接收新的输入写完成控制，并向外透传可写状态。
2. 根据 ready_bank_mask 和 conv_core 空闲状态，选择下一帧启动 bank。
3. 产生内部 launch_pulse，分别送给：
 - conv_core.start
 - conv_shared_input_buffer.start_consume

建议新增端口

端口名	方向	位宽	作用
img_wr_commit	input	1 bit	从顶层透传进来，通知当前写入帧已经写满、可以发车

端口名	方向	位宽	作用
img_wr_ready	output	1 bit	向顶层透传，告知当前输入端是否还能接收一张新图的逐行写入

说明：

- launch_pulse
- launch_bank_sel
- run_enable / continuous_enable

这些更适合作为 **模块内部信号**，不建议作为 conv_subsystem 的对外端口。

为什么这里必须加一层小控制器

因为当前 conv_subsystem.v 是把：

- start

直接同时送给：

- conv_core.start
- conv_shared_input_buffer.start_consume

这在单帧下没问题，但双帧下不够用了，因为现在还需要回答两个问题：

1. 启动哪一个 bank？
2. 什么时候启动下一帧最合适？

如果不在这里做这层控制，你就只能：

- 在 cnn.v 顶层新增 conv_busy 或 start_ready 输出
- 让外部软件自己盯着 Conv 空闲时刻再发 start

这会增加顶层端口和使用复杂度，不符合“端口尽量少”的目标。

推荐做法

建议 conv_subsystem 内部把 start 的语义改成：

- 第一次启动“连续运行模式”或“允许发车”

之后：

- 只要 `conv_core` 空闲
- 且 `ready_bank_mask != 0`

就自动发下一帧。

这样外部只需要：

1. 在 `img_wr_ready=1` 时写 30 行
2. `img_wr_commit`

而不需要关心整个系统的 `busy/done` 时序细节。

4.3 cnn.v

这个模块建议 **只做轻量改动**。

需要增加的功能

1. 增加新的顶层提交/就绪接口并透传给 `conv_subsystem`
2. 不建议在这里增加复杂调度逻辑

建议新增端口

端口名	方向	位宽	作用
<code>img_wr_commit</code>	input	1 bit	顶层输入图整帧写完提交信号
<code>img_wr_ready</code>	output	1 bit	顶层反馈给外部，表示当前输入双缓冲还能接收新图写入

为什么 `cnn.v` 只需要轻改

因为真正的 `bank` 管理和发车判断都应该收敛在 `Conv` 子系统内部。

`cnn.v` 当前主要职责是：

- 连接四个子系统
- 汇总 `busy/done`

如果把“双帧缓存调度器”堆在顶层，会让顶层知道过多 `Conv` 内部细节，反而会扩大改动面。

5. 本轮不建议修改的模块

5.1 conv/conv_core.v

本轮不建议改。

原因：

1. 它已经把输入窗口请求和 token 发射解耦了。
2. 只要 start 变成来自 conv_subsystem 的 launch_pulse，它原有逻辑仍然成立。
3. 当前帧切换时保留 1 到 2 拍启动开销是可以接受的，主要收益来自“隐藏 30 行输入写入时间”。

只有在你后续追求“帧切换完全零空拍”时，才有必要进一步改它的初始化请求逻辑。

5.2 dwconv/*

本轮不建议改。

原因：

- DWConv 没有整帧输入 SRAM
- 接口已经完全是流式 token
- 本轮不需要给它增加 frame_id 或 bank_id

5.3 pwconv/*

本轮不建议改。

原因：

- pwconv_input_buffer.v 已经在 **pos 级** 做了 odd/even 双缓冲
- 它解决的是“同一帧内跨通道聚合”的问题
- 不是“帧级输入图装载”的问题

不要把这两类 Ping-Pong 混在一起。

5.4 post_process/*

本轮不建议作为输入双帧改造的必改项。

尤其是 fc_frame_accum.v：

- 它确实是 README 里提到的另一个潜在优化点
- 但它不属于“输入双帧缓存”这一轮的最小闭环

建议把它放到下一轮“全链路多帧稳态吞吐优化”里单独处理。

6. 推荐的最小端口增量

如果按“最少改动、最少端口”的目标，我建议新增端口如下。

顶层 `cnn.v`

端口名	方向	位宽
<code>img_wr_commit</code>	input	1 bit
<code>img_wr_ready</code>	output	1 bit

`conv_subsystem.v`

端口名	方向	位宽
<code>img_wr_commit</code>	input	1 bit
<code>img_wr_ready</code>	output	1 bit

`conv_shared_input_buffer.v`

端口名	方向	位宽
<code>img_wr_commit</code>	input	1 bit
<code>consume_bank_sel</code>	input	1 bit
<code>ready_bank_mask</code>	output	2 bit
<code>img_wr_ready</code>	output	1 bit

也就是说，真正新增到系统顶层的外部接口，推荐变成：

- 1 根新增输入：`img_wr_commit`

- 1 根新增输出: `img_wr_ready`

而 `img_wr_bank` 则被收敛成内部实现细节。

7. 一个建议的运行时序

推荐的运行流程如下：

1. 外部等待 `img_wr_ready = 1`
2. 外部写入第 0 帧的 30 行数据
3. `img_wr_commit`
4. 输入缓存内部把这张图存到某个 `ready bank`
5. 拉高一次 `start`
6. `conv_subsystem` 选择该 `ready bank` 发车
7. 在当前帧被消费期间，外部继续等待 `img_wr_ready = 1`
8. `img_wr_ready = 1` 后，外部开始写入第 1 帧
9. 写满 30 行后，再拉高一次 `img_wr_commit`
10. 当前帧 Conv 结束后，`conv_subsystem` 自动切到下一张 `ready` 帧

在这个流程里：

- 外部始终只关心“能不能写”
- 内部始终自己决定“写到哪个 `bank` / 读哪个 `bank`”

8. 本方案的边界与注意事项

8.1 不要把“全局 busy”当成下一帧启动条件

当前 `cnn.v` 的：

- `busy = conv_busy || dwconv_busy || pwconv_busy || post_process_busy`

这是“全系统忙”的定义，不是“Conv 输入端还能不能接下一帧”的定义。

双帧输入要看的应该是：

- `conv_core` 是否空闲

- 是否存在 ready bank

所以启动下一帧的判据必须收敛到 `conv_subsystem` 内部。

8.2 最好不要省掉 `img_wr_commit`

理论上你可以用：

- “写到第 29 行”来推断一帧写完

但我不推荐。

因为这会把接口语义绑死在：

- 行必须严格连续写
- 中间不能暂停
- 不能重复写某一行

增加一个 `img_wr_commit` 会更稳，代码也不会多很多。

8.3 采用方案 B 后，必须补一个“可写状态”反馈

因为既然外部不再感知 `img_wr_bank`，那内部就必须告诉外部：

- 现在到底还能不能写下一张图

所以方案 B 下，`img_wr_ready` 这类反馈信号基本是必要的。

否则外部无法判断：

- 当前写进去的数据会不会覆盖未消费的 bank
- 现在是不是应该等待而不是继续送新图

9. 最终建议

如果按“最小可落地闭环”排序，我建议你这样推进：

1. 先只改 `conv_shared_input_buffer.v`、`conv_subsystem.v`、`cnn.v`
2. 先实现“输入双 bank + ready/commit + 自动切换发车”
3. 保持 `conv_core.v` 和所有后续层接口完全不动
4. 先验证“当前帧计算时可并行装下一帧”

5. 再做多样本 back-to-back 仿真

6. 最后再决定要不要单独优化 `fc_frame_accum.v`

一句话概括：

这一轮最优做法不是把整个 CNN 改成 frame-tagged 多帧流水，而是把双帧能力精准地补在 Conv 输入整帧缓存这一处。